

Un catalogue, un bibliothécaire, une réponse à expliquer.

Raconte le même parcours pendant 15 minutes : préparer le catalogue, retrouver les passages, puis rédiger et contrôler la réponse. Chaque fiche traduit l'image en termes techniques et te dit quoi montrer.

État vérifié aujourd'hui

76 tests passent ; Docker et Swagger fonctionnent. La démo directe a été rétablie avec minstral-8b-2512. Les scores de juillet concernent l'ancien modèle. La démo de secours est explicitement enregistrée. Le corpus couvre juillet 2025 → juillet 2026 ; il ne représente pas un agenda actualisé pour aujourd'hui.

COMPRENDRE AVANT DE RÉCITER

Ce que tu as construit, en une minute

Un programme récupère des fiches d'événements, les transforme en passages et en vecteurs, retrouve les passages utiles pour une question, puis demande à Mistral de rédiger une réponse. Une API permet aux autres logiciels de l'utiliser.

Tu n'as pas créé Mistral. Tu as intégré des modèles existants, préparé les données, construit l'index et écrit le code qui relie recherche, réponse et API. Le but était de démontrer une chaîne utilisable par les équipes produit et marketing de Puls-Events.

L'analogie à retenir

Imagine un visiteur qui entre dans une bibliothèque d'événements. Le catalogue contient les fiches OpenAgenda. Les passages sont des extraits de ces fiches. Les embeddings représentent leur sens par des nombres. FAISS joue le bibliothécaire : il compare les vecteurs et retrouve des passages. Le rédacteur Mistral reçoit ces passages et une consigne, puis formule la réponse. FastAPI est le guichet ; Docker transporte l'environnement. FAISS ne raisonne pas comme une personne : l'analogie explique les rôles, sans garantir la compréhension de toutes les contraintes.

Les deux moments du projet

1. **Avant les questions** : collecte → nettoyage → Parquet → découpage → embeddings Mistral → sauvegarde FAISS. Cette préparation appelle déjà internet.
2. **À chaque question** : embedding Mistral de la question → recherche locale de quatre passages → contexte + prompt → chat Mistral → JSON avec réponse et sources.

Les chiffres qui correspondent vraiment au code

À mémoriser	Ce que cela veut dire
7 586 événements	7 586 identifiants uniques dans le corpus livré, tous représentés dans les chunks.
14 903 passages	Nombre de chunks de tout le corpus, égal au nombre de vecteurs FAISS.
1 000 / 200 caractères	Taille de découpage / chevauchement maximal. Ce ne sont pas des tokens.
1 024 dimensions	1 024 nombres par vecteur, vérifiés dans index.faiss.
top-k = 4	Quatre passages recherchés ; potentiellement moins de quatre événements après déduplication des sources.
76 tests	Suite vérifiée le 1er octobre : 76 tests réussis avec des doublures ; appels réels vérifiés séparément.
4 cas locaux / 3 cas RAGAS	Deux jeux distincts et deux méthodes distinctes.
0,900 / 0,846 / 1,000 / 0,683	Fidélité / pertinence / précision du contexte / correction, sur trois cas le 23 juillet.

Les six étapes demandées, et où tu les retrouves

Étape	Ce que fait le code	Fichiers à connaître
1 · Environnement	Python 3.12, versions directes des dépendances, variables d'environnement.	environment.yml requirements.txt scripts/check_env.py
2 · Collecte	Filtre ville/date, parcourt les pages, nettoie HTML et champs, exporte JSON/Parquet/manifeste.	scripts/fetch_events.py src/openagenda_rag/ingestion.py
3 · Index	Découpe, calcule les embeddings par lots, construit et persiste FAISS avec métadonnées.	scripts/build_index.py src/openagenda_rag/indexing.py
4 · Chatbot	Retrouve les chunks, compose le prompt, appelle le chat et sérialise la réponse.	scripts/chatbot.py src/openagenda_rag/rag.py src/openagenda_rag/service.py
5 · API	Valide le JSON, appelle le service, expose les routes et Swagger.	scripts/run_api.py src/openagenda_rag/api.py

Étape	Ce que fait le code	Fichiers à connaître
		y scripts/api_test.py
6 · Livraison	Image, volume, seed et lancement local ; scripts de démonstration et supports.	Dockerfile docker-compose.yml scripts/docker_entrypoint.sh scripts/demo_api_5min.py
Validation transversale	Tests du code, cas annotés, règles locales et quatre métriques RAGAS.	tests/ scripts/evaluate_rag.py scripts/evaluate_ragas.py

Un événement réel de bout en bout

Concert Fishers est la fiche 57857032. Le texte contient notamment « reprises pop, rock, funk ». Les métadonnées donnent Paris, Le Gymnase Montparnasse, le 21 juin 2026 de 19 h 30 à 23 h 30, et l'URL OpenAgenda. Le chunk 57857032 : : chunk-0 possède un vecteur. La question « Parle-moi de Concert Fishers a Paris » permet de le retrouver et de fournir ces faits au modèle. Ce concert est passé au jour de la soutenance ; il sert à vérifier la chaîne sur le corpus livré.

Le parcours pour te préparer

1. **15 min** : lis « Comprendre » et le glossaire ; explique à voix haute la différence embedding / FAISS / chat.
2. **25 min** : lis le script des 12 slides dans l'ordre, avec les explications simples. Ne cherche pas à apprendre tous les détails techniques.
3. **15 min** : répète la démo avec la page dédiée ; apprends à annoncer le mode enregistré et à montrer les sources.
4. **15 min** : lis les réponses sur le corpus, top-k, les scores et les limites. Réponds d'abord en une phrase.
5. **15 min** : lance le chronomètre du deck et fais une répétition complète. Les durées incluent les changements d'écran, la démo et la conclusion sur le support.
6. **Avant 14 h** : vérifie son, partage de fenêtre, liens, Docker et mode secours. Garde de la marge pour te connecter.

Démo directe rétablie : modèle à annoncer

Mistral Small renvoyait 429 alors que les embeddings fonctionnaient. La génération a été basculée sur ministral-8b-2512 et les deux scénarios ont réussi en direct. Annonce ce changement ; les scores RAGAS de juillet restent ceux de l'ancien modèle. Compare toujours la sortie aux sources.

Écran partagé et écran personnel

Partage la **fenêtre** du diaporama ou de la démo. Ce guide contient tes notes et doit rester hors partage. Si tu n'as qu'un écran, place deux fenêtres côte à côte et partage la bonne fenêtre. Évite de partager tout le bureau. Utilise un second écran ou ce guide imprimé si disponible.

Le contrat de la soutenance

15 minutes de livrables, 10 minutes de discussion, 5 minutes de débrief. Le jury joue Jérémy pendant les deux premières parties. La soutenance porte sur la mission Puls-Events, pas sur les exercices municipaux du cours également présents dans le document exporté. Cible 15 minutes ; en dessous de 10 ou au-dessus de 20 minutes, la présentation peut être refusée.

Ce qui est prêt, ce qui doit rester explicite

Vérifié aujourd'hui

76 tests passent. Docker, /health et Swagger répondent. Les deux scénarios /ask ont réussi avec ministral-8b-2512. Les volumes et dimensions de FAISS sont vérifiés.

Preuves historiques

Réponses de démo enregistrées ; RAGAS du 23 juillet. Le corpus a été collecté le 10 juillet. Les deux scénarios ont aussi réussi en direct aujourd'hui avec Ministral 8B ; RAGAS n'a pas été recalculé.

La question « explique-moi ton projet »

« J'ai construit une bibliothèque d'événements pour Puls-Events. Le catalogue contient les fiches OpenAgenda. Je prépare leurs passages et leurs représentations du sens. Pour une question, FAISS joue le bibliothécaire et retrouve quatre passages. Mistral les reçoit pour rédiger une réponse que l'on peut vérifier dans les fiches. FastAPI ouvre un guichet aux logiciels de vos équipes. J'ai testé la mécanique et évalué des réponses ; le catalogue doit être actualisé et les garde-fous renforcés avant production. »

Les mots que tu dois pouvoir expliquer

POC

Prototype qui démontre la faisabilité, avec un périmètre limité.

LLM

Le rédacteur de notre bibliothèque : un modèle de langage qui formule une réponse à partir du contexte.

Catalogue / corpus

L'ensemble des fiches disponibles pour la recherche ; ici 7 586 événements de Paris collectés en juillet.

RAG

Le parcours chercher puis rédiger : des passages du catalogue sont fournis au modèle au moment de la question.

Chunk

Un passage découpé dans une fiche du catalogue ; il reste relié à son événement.

Embedding / vecteur

Une représentation du sens par des nombres, pour comparer passages et question ; elle ne garantit pas que toutes les contraintes sont comprises.

Dimension

Nombre de coordonnées dans le vecteur : 1 024 dans l'index livré.

FAISS

Le bibliothécaire de l'analogie : une bibliothèque qui recherche les vecteurs voisins et retourne leurs passages ; elle ne rédige pas.

IndexFlatL2

Index exact qui compare les distances euclidiennes au carré ; distance faible = vecteurs proches.

Retriever / top-k

Sélecteur des k passages les plus proches ; k vaut 4 ici.

Contexte / prompt

Les extraits déposés sur le bureau du rédacteur et la consigne qui lui indique comment répondre.

Métadonnées

Informations autour du texte : identifiant, ville, lieu, dates, URL...

LangChain

Briques Python pour manipuler documents, découpage, recherche et appels aux modèles.

API REST / endpoint

Le guichet de la bibliothèque : un contrat HTTP qui permet à un logiciel d'appeler une route comme POST /ask.

JSON

Format des messages de l'API : champs question, answer, sources...

Parquet

Format de tableau compact utilisé pour les données normalisées.

Seed / manifeste

Le seed est le corpus livré ; le manifeste trace sa source, ses paramètres et ses volumes.

Docker / volume

Environnement conteneurisé ; le volume garde les fichiers entre deux créations du conteneur.

Température / token

Température : réglage de variation du texte. Token : unité de texte du modèle, différente d'un caractère.

RAGAS

Un outil pour examiner les extraits et les réponses avec des modèles juges et des références ; ici trois cas historiques.

Doublure / mock

Service simulé pour tester le code sans appels externes.

CI/CD

Automatisation des validations et déploiements ; elle est proposée, pas livrée ici.

Le conducteur unique

Les durées totalisent 15 minutes, avec la démo dans la slide 8 et la conclusion sur le support dans la slide 12. Le texte est un conducteur : parle posément, montre les preuves et marque les transitions. Une lecture sans gestes sera plus courte ; répète avec le chronomètre.

Slide	Rôle	Repère	Durée
01	Accueillir la demande	00:00 → 00:45	45 s
02	Catalogue → bibliothécaire → rédaction	00:45 → 01:50	65 s
03	Les fiches OpenAgenda	01:50 → 03:05	75 s
04	Passages et embeddings	03:05 → 04:20	75 s
05	FAISS et modèles Mistral	04:20 → 05:30	70 s
06	Avant et pendant la question	05:30 → 06:30	60 s
07	FastAPI et Docker	06:30 → 07:35	65 s
08	Démo en direct	07:35 → 09:55	140 s
09	Tests et jeu annoté	09:55 → 11:00	65 s
10	Comprendre les scores RAGAS	11:00 → 12:40	100 s
11	Limites et améliorations	12:40 → 13:45	65 s
12	Livraison et perspectives	13:45 → 15:00	75 s

Après cela : 10 minutes de discussion, puis 5 minutes de débrief. Ne lis pas les explications techniques pendant les slides ; elles sont là pour comprendre et répondre.

Un catalogue culturel, une réponse vérifiable

L'idée à retenir : Le visiteur pose une question ; nous cherchons dans des fiches réelles avant de répondre.

Bonjour Jérémie. Pour expliquer ce POC, je vous propose une image simple : une bibliothèque d'événements culturels. Un visiteur arrive avec une demande, par exemple une sortie en famille à Paris. Il souhaite une réponse utile et des informations qu'il peut vérifier. Mon travail a été de constituer le catalogue, de préparer ses fiches pour la recherche, puis de relier cette recherche à un modèle qui rédige et à une API que vos équipes peuvent appeler. Le catalogue livré contient 7 586 événements parisiens. Je vais suivre ce parcours avec vous, montrer deux demandes en direct et expliquer ce qui fonctionne ainsi que ce qui reste à améliorer.

À montrer / à faire

Partage uniquement la fenêtre du diaporama. Garde ce guide hors du partage.

▼ Comprendre simplement

Le catalogue est réel. Le bibliothécaire est une image pour expliquer la recherche, pas une personne ni un modèle entraîné sur tes données.

▼ Si le jury creuse la technique

Le brief demande LangChain, Mistral, FAISS, une API, les scripts de reconstruction, des tests, un jeu annoté, un rapport, Docker et une démo. L'historique de conversation n'est pas nécessaire. Le modèle n'a pas été entraîné ou ajusté dans ce projet.

▼ La preuve dans le projet

README.md ; document de mission, message de Jérémie ; seed-data/processed/fetch_manifest.json.

Le piège à éviter

Ne dis pas que c'est déjà un produit en production ni que tu as entraîné Mistral.

Transition à dire : « Pour comprendre la solution, séparons le catalogue, la recherche et la rédaction. »

Chercher dans le catalogue avant de répondre

L'idée à retenir : Le RAG apporte des passages au rédacteur au moment de la question.

Dans notre bibliothèque, les fiches OpenAgenda forment le catalogue. Chaque fiche décrit un événement, avec son titre, son lieu, ses dates et son lien. Le bibliothécaire représente la recherche avec FAISS : il retrouve des passages proches de la demande. Puis un rédacteur, le modèle Mistral, reçoit ces passages et formule la réponse. Les embeddings sont ce qui permet de comparer numériquement le sens des textes et de la question. Le RAG consiste à chercher d'abord, puis à rédiger avec les informations retrouvées. Je n'ai pas entraîné Mistral sur le catalogue : j'ai construit l'index et organisé ce passage d'informations. L'analogie a une limite : FAISS compare des vecteurs, il ne comprend pas toutes les contraintes comme un bibliothécaire humain.

À montrer / à faire

Pointe les trois étapes. L'exemple famille existe dans le jeu RAGAS ; ce n'est pas une nouvelle preuve live.

▼ Comprendre simplement

Catalogue = corpus ; passages = chunks ; représentation du sens = embeddings ; bibliothécaire = recherche FAISS ; rédacteur = modèle de chat.

▼ Si le jury creuse la technique

Ce n'est ni du fine-tuning ni de l'entraînement. L'embedding représente le texte numériquement, le retriever sélectionne des documents et ChatPromptTemplate assemble les messages transmis au chat. Les documents sont consultés à chaque question.

▼ La preuve dans le projet

src/openagenda_rag/rag.py : answer_question(), build_prompt() ; tests/fixtures/ragas_eval_dataset.csv.

Le piège à éviter

Ne promets pas zéro hallucination. Une recherche proche n'est pas nécessairement une réponse pertinente.

Transition à dire : « Avant d'aider un visiteur, il faut constituer un catalogue fiable. »

Constituer et nettoyer le catalogue

L'idée à retenir : Conserver des fiches lisibles et leurs références pour vérifier les réponses.

J'ai commencé par collecter les fiches du dataset public OpenAgenda indiqué par la mission, accessible via Opendatasoft. J'ai choisi Paris et une fenêtre du 10 juillet 2025 au 10 juillet 2026. La collecte parcourt les pages, retire les doublons par identifiant et nettoie les descriptions, notamment les balises HTML. Je garde les informations qui permettent de vérifier une réponse : titre, description, lieu, dates et URL. Les données sont exportées dans un fichier Parquet, un format structuré relisible par les scripts. Ce catalogue contient 7 586 événements. Comme une bibliothèque dont le catalogue n'a pas été mis à jour depuis juillet, il manque les nouvelles fiches d'octobre. Je reconnais cette limite : pour conseiller des sorties actuelles, il faut actualiser les données et vérifier les dates.

À montrer / à faire

Reste sur la diapositive : montre Paris, la période du catalogue et les trois opérations de nettoyage.

▼ Comprendre simplement

Nettoyer le catalogue ne signifie pas inventer ce qui manque. Tu conserves les informations disponibles et les références. Le catalogue est un instantané de juillet.

▼ Si le jury creuse la technique

`build_records_where_clause()` filtre `firstdate_begin`, pas chaque occurrence récurrente. `last_timing` peut dépasser juillet 2026. La borne haute fixe exclut les événements qui commencent après la collecte. Les données ne sont pas automatiquement rafraîchies. 39 titres sont absents dans le seed, mais leurs descriptions peuvent rester indexables. Déduplication par `event_uid` ; les titres identiques avec des UID différents ne sont pas fusionnés.

▼ La preuve dans le projet

`scripts/fetch_events.py` ; `src/openagenda_rag/ingestion.py` ; `seed-data/processed/fetch_manifest.json` ; `events.parquet`.

Le piège à éviter

La consigne demande des événements récents et vise des recommandations à venir. Au 1er octobre, certains événements du seed ont plus d'un an. La fraîcheur constitue donc un écart actuel à signaler et à corriger par une nouvelle collecte, pas à masquer.

Transition à dire : « Une fiche peut être longue : nous allons préparer des passages plus faciles à retrouver. »

03:05 → 04:20 • 75 secondes • repère incluant gestes et pauses

Découper les fiches, puis comparer leur sens

L'idée à retenir : Une fiche longue devient plusieurs passages, chacun relié à son événement.

Dans un catalogue, certaines fiches sont courtes et d'autres très longues. Pour ne pas transmettre toute une longue description au rédacteur, je les découpe en passages, appelés chunks. Le réglage de départ est de 1 000 caractères, avec jusqu'à 200 caractères de chevauchement pour préserver les informations aux frontières. Le découpage cherche à respecter les séparateurs du texte. La fiche CHIMERE illustre ce point : elle produit 15 passages. Tout le catalogue produit 14 903 passages, et non 14 903 événements. Chaque passage est transformé par mistral-embed en un vecteur de 1 024 nombres. Ces nombres servent à comparer le sens ; ce ne sont pas des catégories lisibles par un humain. Je conserve aussi le lien vers la fiche, ses dates et son lieu.

À montrer / à faire

Distingue la fiche d'exemple et les volumes de tout le corpus. Aucun réindexage en direct.

▼ Comprendre simplement

Le passage est le texte. Le vecteur est sa représentation numérique. Les métadonnées sont sa référence dans le catalogue. Ce sont trois choses différentes.

▼ Si le jury creuse la technique

RecursiveCharacterTextSplitter, add_start_index=True. Le chevauchement est un paramètre maximal, pas 200 caractères garantis à chaque frontière. chunk_id = event_uid::chunk-N. L'index réel est IndexFlatL2 : recherche exacte selon la distance euclidienne au carré, sans normalisation explicite dans le code. Moins la distance est élevée, plus les vecteurs sont proches. Tous les 7 586 UID sont représentés. CHIMERE 69427795 a 15 chunks ; une autre fiche porte le même titre avec un UID différent.

▼ La preuve dans le projet

src/openagenda_rag/indexing.py : build_documents(), build_vector_store() ; seed-data/index/index_manifest.json ; index.faiss.

Le piège à éviter

Caractères et tokens sont différents. Ne dis pas que CHIMERE seul produit 14 903 chunks. Un vecteur n'est ni un résumé ni une réponse.

Transition à dire : « Voyons comment le bibliothécaire utilise ces représentations pour une question. »

Le bibliothécaire cherche ; le rédacteur répond

L'idée à retenir : Quatre passages retrouvés deviennent le contexte du rédacteur.

Quand le visiteur pose une question, mistral-embed la transforme avec le même modèle que les passages. Les vecteurs deviennent ainsi comparables. FAISS retrouve les quatre passages les plus proches : c'est le top-k fixé à quatre. Dans notre image, le bibliothécaire dépose quatre extraits sur le bureau du rédacteur. Le prompt est sa consigne : répondre en français à partir de ces extraits, citer les informations disponibles et dire qu'il ne sait pas si elles manquent. Mistral Small était le modèle initial. Aujourd'hui, il renvoyait une limite d'usage ; j'ai configuré la version ministral-8b-2512, puis vérifié la démo. L'embedding reste identique, donc le catalogue vectoriel n'a pas été reconstruit. Les scores historiques concernent toutefois Small. Le prompt aide le rédacteur, mais ne garantit pas une réponse sans erreur.

À montrer / à faire

Pointe la séparation embedding/chat et explique les consignes du rédacteur sur le support.

▼ Comprendre simplement

FAISS choisit les passages ; le modèle de chat rédige. Quatre passages ne signifient pas quatre événements. La température 0,1 et la limite de 700 tokens sont des réglages de départ, pas un optimum démontré.

▼ Si le jury creuse la technique

`answer_question()` appelle `retrieve.invoke()`, `build_source_entries()`, puis `prompt.invoke()` et `chat_model.invoke()`. Les sources sont les événements récupérés, pas une validation que chaque source soutient chaque phrase. Aucun filtre de date ou de ville n'est appliqué dynamiquement dans le `retrieve`. L'absence de documents déclenche un refus déterministe ; en recherche top-k, le résultat non vide est fréquent même hors sujet. `random_seed=42` stabilise sans garantir une sortie identique. L'alias `latest` peut évoluer. Le modèle de démonstration est fixé à `ministral-8b-2512` dans `.env`. L'embedding reste `mistral-embed` : l'index est compatible sans reconstruction. Les scores RAGAS historiques concernent `mistral-small-latest` et ne valident pas ce changement. La date du jour et un statut temporel calculé sont transmis au prompt ; aucun filtre de date ne supprime les passages.

▼ La preuve dans le projet

`src/openagenda_rag/rag.py` ; `src/openagenda_rag/settings.py` : `load_rag_settings()`.

Le piège à éviter

Ne dis pas que les sources renvoyées sont toutes utilisées dans la réponse ni que le refus Lyon est codé comme une règle de ville.

Transition à dire : « Cette bibliothèque fonctionne en deux temps : la préparation et le traitement d'une demande. »

Préparer les rayons une fois, chercher à chaque demande

L'idée à retenir : La préparation du catalogue et la réponse à une question sont deux étapes distinctes.

Il faut distinguer les deux moments. Avant les questions, je prépare les rayons : collecte, nettoyage, découpage, calcul des embeddings et enregistrement de l'index FAISS. Chaque passage reste relié à sa fiche. Puis, à chaque demande, je vectorise uniquement la question, recherche les passages et appelle le rédacteur. Je ne recollecte pas tout OpenAgenda et je ne recalculerai pas tous les vecteurs à chaque question. Le fichier FAISS livré contient 14 903 vecteurs ; tous les identifiants d'événements sont représentés. L'index est chargé à la première demande, puis réutilisé. Attention, les rayons sont locaux, mais mistral-embed et le rédacteur sont des services distants. Cette bibliothèque dépend donc encore d'internet.

À montrer / à faire

Pointer les deux moments sur le support : préparer les rayons une fois ; rechercher et répondre à chaque demande.

▼ Comprendre simplement

Construire l'index, c'est préparer les rayons. Interroger l'index, c'est chercher dedans. Docker contient le seed ; le volume conserve l'index de travail.

▼ Si le jury creuse la technique

OpenAgendaRAGService._ensure_runtime() initialise les objets paresseusement. Ce sont des clients et un retriever en mémoire, pas les poids de Mistral en local. rebuild() reconstruit depuis le Parquet et invalide ces objets ; il ne recollecte pas les événements. La CLI réutilise le service ; la génération reste distante même sans HTTP local.

▼ La preuve dans le projet

src/openagenda_rag/service.py ; scripts/chatbot.py ; scripts/run_api.py.

Le piège à éviter

Ne qualifier pas la préparation de complètement hors ligne : elle appelle Mistral. /health ne charge pas l'index et ne teste pas la clé.

Transition à dire : « Pour que vos équipes puissent poser une question, il reste à ouvrir un guichet. »

Un guichet pour les équipes produit

L'idée à retenir : L'API ouvre un accès au catalogue, à la recherche et à la rédaction.

FastAPI est le guichet de notre bibliothèque. Une application lui envoie une question en JSON et reçoit une réponse, des sources et le nombre de passages récupérés. La route ask fait ce travail. Health indique que le guichet répond et affiche sa configuration, mais ne vérifie pas à elle seule que le bibliothécaire et le rédacteur peuvent répondre. Rebuild reconstruit les rayons à partir du Parquet ; elle ne collecte pas de nouvelles fiches. Swagger documente ces routes. Docker rassemble le code, les dépendances et le catalogue de départ pour lancer le même service ailleurs. La clé Mistral reste côté serveur. Le jeton de reconstruction est optionnel dans le POC ; il faut sécuriser ce guichet avant de l'exposer publiquement.

À montrer / à faire

Reste sur le schéma du guichet API. Explique les routes sans ouvrir Swagger ; la démonstration se fait dans la page dédiée à la slide suivante.

▼ Comprendre simplement

API = guichet accessible par un logiciel. JSON = format du message. Docker = environnement transportable ; il ne rend pas Mistral local.

▼ Si le jury creuse la technique

Validation Pydantic : question vide → 422 ; espaces seuls → 400 ; index absent → 503. Les erreurs HTTP Mistral 429 et d'accès sont contrôlées en 503, les autres erreurs HTTP fournisseur en 502, les erreurs réseau en 503. Les détails des exceptions fournisseur et inattendues ne sont plus exposés. /rebuild est protégé uniquement si API_REBUILD_TOKEN est défini. /health vérifie le serveur et sa configuration, sans appeler Mistral.

▼ La preuve dans le projet

src/openagenda_rag/api.py ; Dockerfile ; docker-compose.yml ; scripts/docker_entrpoint.sh.

Le piège à éviter

Ne dis pas que /rebuild est forcément protégé ou que /health valide Mistral et l'index.

Transition à dire : « Passons maintenant au guichet avec deux demandes concrètes. »

Au guichet : une fiche trouvée, une demande hors catalogue

L'idée à retenir : Observer le trajet d'une vraie demande, puis contrôler la réponse avec la fiche.

Je reste dans la page de démonstration et je vérifie d'abord l'état du guichet avec le bouton health. Ce contrôle vérifie la configuration du serveur ; l'appel suivant montrera si toute la chaîne répond. Dans la page de démonstration, je demande des informations sur Concert Fishers à Paris et je clique sur l'appel direct. La page transmet la question à l'API, qui recherche les passages et appelle le rédacteur. Je compare maintenant la réponse à la fiche : Concert Fishers, Le Gymnase Montparnasse, le 21 juin 2026 et l'URL OpenAgenda. Cet événement est passé ; il sert à vérifier la chaîne sur le catalogue livré. Pour la deuxième demande, je cherche des expositions photo à Lyon. Notre catalogue couvre Paris. Le modèle indique qu'il ne sait pas répondre à partir de ce corpus. Le bibliothécaire retrouve tout de même des passages voisins : ce ne sont pas des recommandations pour Lyon. Cette distinction explique pourquoi un filtre de ville serait utile.

À montrer / à faire

140 s au total : 20 s bouton health dans la page de démo ; 50 s cas nominal ; 40 s Lyon ; 30 s explication et retour au diaporama. Une tentative live maximum en cas de quota, puis enregistrement annoncé.

▼ Comprendre simplement

La page est une interface HTML ajoutée pour la soutenance. Elle appelle réellement le guichet FastAPI. Son bouton enregistré montre une répétition sauvegardée, explicitement séparée du direct.

▼ Si le jury creuse la technique

Le 1er octobre, les deux scénarios /ask ont réussi en direct avec minstral-8b-2512. Le rapport outputs/demo/demo_api_2026-10-01.json en conserve les réponses, le modèle et l'horodatage. La mesure couvre health et les deux questions, pas chaque requête ni un benchmark. Mistral Small renvoyait 429 ; le modèle de génération a été changé, sans modifier mistral-embed ni reconstruire FAISS. La sortie peut encore contenir un conseil général non sourcé : le signaler si présent.

▼ La preuve dans le projet

scripts/demo_api_5min.py ; outputs/demo/demo_api_2026-10-01.json ; démonstrateur HTML ; état de préparation.

Le piège à éviter

Ne présente pas un enregistrement comme un appel direct. Compare les faits et les dates aux sources. Si un conseil général non sourcé apparaît, explique que le prompt réduit les erreurs sans les éliminer. Aucun secret à l'écran.

Transition à dire : « Deux demandes donnent des exemples. Comment vérifier plus largement la bibliothèque ? »

Tester le mécanisme et examiner les réponses

L'idée à retenir : Les tests du guichet et des rayons ne suffisent pas à juger toutes les réponses.

Pour contrôler la bibliothèque, je distingue la mécanique et la qualité des conseils. Les 76 tests locaux passent : ils vérifient notamment les filtres, le nettoyage, le découpage, l'indexation, le contrat API et les erreurs. Ils utilisent des doublures pour ne pas dépendre du fournisseur à chaque exécution. Les appels réels sont vérifiés séparément dans la démo. Le jeu fonctionnel contient quatre questions annotées, avec des titres attendus et un cas de refus. Sa règle est permissive : un titre présent dans les sources peut suffire à classer le cas comme correct, même si la réponse reste imparfaite. Quatre cas corrects ne signifient donc pas cent pour cent de fiabilité. Aucune chaîne CI/CD n'est livrée ; les scripts permettent déjà d'automatiser les vérifications.

À montrer / à faire

Reste sur la diapositive : annonce les 76 tests et explique la différence avec les quatre questions annotées.

▼ Comprendre simplement

Un guichet qui répond et des rayons accessibles ne prouvent pas que le rédacteur donne toujours le bon conseil. Les tests du code et l'évaluation des réponses sont complémentaires.

▼ Si le jury creuse la technique

evaluation.py utilise SequenceMatcher sur du texte normalisé : similarité lexicale, pas embedding sémantique. exact_match_soft = égalité après normalisation. matched_expected_titles cherche dans answer OU sources. Sans titre attendu, certaines expressions de refus suffisent à classer correct. Les sorties historiques ont 4 corrects, exact_match_soft_rate=0 et similarité moyenne ~0,131. Ce n'est pas contradictoire, car les mesures ne testent pas la même chose. --ragas sur evaluate_rag.py ne lance pas les quatre métriques ; le vrai script est evaluate_ragas.py.

▼ La preuve dans le projet

tests/ ; tests/fixtures/rag_eval_dataset.csv ; scripts/evaluate_rag.py ; outputs/evaluation/latest_summary.json.

Le piège à éviter

Ne transforme pas 4/4 en une précision de 100 % du système. Les tests ne font pas tous des appels Mistral réels.

Transition à dire : « RAGAS apporte une seconde lecture : examiner les extraits et le travail du rédacteur. »

Le lecteur contrôle les extraits et la rédaction

L'idée à retenir : Quatre métriques apportent quatre regards différents sur trois réponses.

J'ai aussi utilisé RAGAS sur trois cas, avec les passages effectivement récupérés, la réponse et une référence humaine. Reprenons la bibliothèque. La fidélité demande si les affirmations du rédacteur sont soutenues par les extraits : sa moyenne est 0,900. La pertinence demande si la réponse traite la demande du visiteur : 0,846. La précision du contexte examine l'utilité et l'ordre des extraits apportés par le bibliothécaire : 1,000. Ce score ne signifie pas qu'il a retrouvé tous les événements pertinents. La correction compare la réponse à la référence attendue : 0,683. Une réponse peut donc être cohérente avec les extraits, tout en oubliant une information ou en s'éloignant de la référence. Le cas famille est le plus faible en correction, autour de 0,581. Ces résultats datent du 23 juillet, utilisent Mistral Small et ne portent que sur trois cas. Ils orientent les améliorations ; ils ne démontrent pas la qualité générale de Ministral 8B.

À montrer / à faire

Reste sur les quatre scores : explique leur sens, l'exemple de correction faible et les limites du petit échantillon.

▼ Comprendre simplement

Fidélité = appui sur les extraits. Pertinence = réponse à la demande. Précision du contexte = utilité des extraits. Correction = accord avec la référence. Ce sont des scores du petit échantillon, pas des pourcentages de satisfaction.

▼ Si le jury creuse la technique

RAGAS 0.4.3 ; juge ministral-8b-2512 ; embeddings mistral-embed. Faithfulness juge les affirmations soutenues. ContextPrecision s'appuie sur une référence pour évaluer les contextes récupérés et leur classement : 1,0 ne signifie ni tous les chunks utiles ni rappel parfait. AnswerCorrectness combine comparaison factuelle et similarité. Le script appelle les métriques séquentiellement avec reprises après erreurs transitoires. Les seuils fort $\geq 0,8$ et acceptable $\geq 0,6$ sont des conventions du projet. Le même fournisseur sert à générer et juger : biais possible, review humaine nécessaire. Les références courtes peuvent pénaliser des développements utiles, mais cela ne justifie pas les faits inventés.

▼ La preuve dans le projet

scripts/evaluate_ragas.py ; src/openagenda_rag/ragas_evaluation.py ; outputs/evaluation/ragas_results.json ; rapport, section 7.

Le piège à éviter

0,900 n'est pas un taux de satisfaction de 90 %. 1,000 n'est pas un rappel de 100 %. Ces scores historiques n'ont pas été recalculés aujourd'hui.

Transition à dire : « Ces contrôles mettent en évidence trois limites concrètes de notre bibliothèque. »

Un catalogue daté, une recherche imparfaite, un rédacteur externe

L'idée à retenir : L'analogie permet aussi de comprendre où le système peut échouer.

La première limite concerne le catalogue : il est limité à Paris et figé en juillet. Il faut actualiser les fiches et filtrer les occurrences pour recommander des sorties actuelles. La deuxième concerne le bibliothécaire : il apporte les passages les plus proches, même lorsque leur ville ou leur date ne répond pas à la demande. Un seuil calibré et des filtres explicites rendraient le refus plus robuste. La troisième concerne le rédacteur : il dépend de Mistral, peut rencontrer une limite d'usage et peut encore ajouter un conseil hors contexte. J'ai rétabli la démo avec Minstral 8B et contrôlé les erreurs fournisseur, mais cela ne prouve pas une qualité équivalente à Small. Il faut réévaluer ce modèle, diversifier les questions et renforcer les contrôles avant un déploiement plus large.

À montrer / à faire

Relie chaque limite à une amélioration. Si la démo enregistrée a été utilisée, rappelle simplement la cause observée.

▼ Comprendre simplement

Catalogue incomplet → données à actualiser. Mauvais extraits → recherche à calibrer. Conseil fragile → réponse à vérifier. L'analogie explique les responsabilités sans masquer les limites.

▼ Si le jury creuse la technique

Pas de reranker, recherche hybride, seuil de distance, filtre de métadonnées par question, authentification globale, limitation de débit ni supervision livrés. Reconstruction destructive sur le répertoire cible : pas de bascule atomique et risque de concurrence à traiter en production. Pickle de FAISS chargé avec `allow_dangerous_deserialization=True` : uniquement artefacts de confiance. Les noms latest et les dépendances transitives non totalement figées limitent la reproductibilité stricte.

▼ La preuve dans le projet

README.md : limites ; src/openagenda_rag/rag.py ; api.py ; indexing.py ; état de préparation.

Le piège à éviter

Ne prétends pas que Docker rend le projet entièrement hors ligne, qu'un seuil seul comprend toutes les contraintes, ou que l'API est prête pour une exposition publique.

Transition à dire : « La prochaine étape est donc d'améliorer cette bibliothèque et de mesurer son utilité avec vos équipes. »

13:45 → 15:00 • 75 secondes • repère incluant gestes et pauses

Une bibliothèque testable ; un pilote à mesurer

L'idée à retenir : Le POC livre le parcours complet ; le service doit encore être amélioré et évalué.

Pour conclure, reprenons le trajet du visiteur dans notre bibliothèque. Il pose une question ; le guichet la reçoit ; sa représentation numérique permet au bibliothécaire de retrouver les passages proches ; le rédacteur les utilise pour formuler une réponse, accompagnée des fiches à vérifier. Le POC réalise ce parcours avec un catalogue réel, une recherche locale et des modèles existants. La démonstration a montré ce qui fonctionne, mais aussi pourquoi une source retrouvée n'est pas toujours adaptée à la demande. Pour aller plus loin, je propose trois priorités. D'abord, actualiser le catalogue et appliquer les contraintes de ville et de date. Ensuite, élargir les références humaines pour comparer les réglages et les modèles sur davantage de situations, y compris les demandes sans réponse. Enfin, mesurer un pilote avec les équipes : utilité des réponses, refus corrects, clics vers les sources, latence et coût. Les tests vérifient la mécanique ; l'évaluation et les utilisateurs doivent vérifier son utilité. La bibliothèque est testable, et sa qualité à grande échelle reste à mesurer. Je suis prêt à répondre à vos questions.

À montrer / à faire

Reste sur la conclusion pendant 75 s : rappelle le parcours de la bibliothèque, relie les trois améliorations aux limites, puis ouvre la discussion. Aucun changement vers GitHub, le code ou le rapport n'est prévu.

▼ Comprendre simplement

Ta conclusion : une chaîne concrète est livrée, des preuves existent et les prochains travaux répondent aux limites observées. Pas de promesse de production ou de qualité parfaite.

▼ Si le jury creuse la technique

Priorités : fraîcheur et contraintes métier ; jeu annoté diversifié et cas négatifs ; optimisation mesurée ; gestion quotas/erreurs/secrets ; authentification, logs, CI, tests de charge et mises à jour atomiques. Les clics et la satisfaction doivent être observés avec des utilisateurs. Aucun coût précis ni objectif de latence de production n'a été mesuré dans ce POC.

▼ La preuve dans le projet

README.md ; outputs/rapport-technique-openagenda-rag.pdf ; <https://github.com/hgbe-GH/Projet7>.

Le piège à éviter

Garde la conclusion sur le support. Ne promets ni une qualité parfaite ni un service prêt pour la production. Le rapport et le dépôt sont disponibles si le jury les demande ; leur visite n'est pas intégrée à ton récit.

Transition à dire : « Discussion : commence par la réponse courte, puis explique le composant et la preuve concernés. »

La démo, geste par geste

Tu as **2 min 20** pour la démo et son interprétation. [Ouvrir le démonstrateur](#) · [Swagger](#).

1. **20 s** : clique sur le bouton « Vérifier /health » dans la page de démonstration. Dis « Le serveur et le contrat sont disponibles ; health ne prouve pas à lui seul que Mistral répond. »
2. **50 s** : sélectionne Concert Fishers. Si le mode direct fonctionne, appelle /ask. Sinon, annonce l'échec et clique explicitement « Réponse enregistrée ». Montre date, lieu, URL et premier événement source.
3. **40 s** : sélectionne Lyon. Montre le refus enregistré ou reçu en direct et la ville Paris dans les sources. Explique que ces sources sont récupérées, sans être des recommandations adaptées à Lyon.
4. **30 s** : dis la limite : pas de seuil ni filtre ville dans le retrieveur. Retourne au deck, slide 9.

Si Mistral reste limité aujourd'hui

« L'API locale est disponible, mais le fournisseur Mistral renvoie actuellement une limite d'usage. Je vous montre le contrat et les résultats d'une répétition réelle enregistrée, que je distingue d'un appel live. Le résultat permet d'examiner le fonctionnement, mais l'indisponibilité externe empêche la démonstration de bout en bout en direct dans cette session. »

Dans la page de démonstration, aucun échec direct n'est remplacé automatiquement par un succès enregistré. La date et la provenance sont affichées. Les enregistrements existent localement et restent consultables sans internet.

Alternative par terminal

```
curl -s http://127.0.0.1:8000/health

curl -s -X POST http://127.0.0.1:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "Parle-moi de Concert Fishers a Paris"}'
```

Ne montre pas un détail d'erreur brut. Ne lance pas /rebuild, une collecte complète ou RAGAS pendant la soutenance. Le script demo_api_5min.py s'arrête désormais au premier échec ; la page permet aussi de contrôler une tentative à la fois.

Ce qu'il faut lire dans le JSON

Champ	Interprétation
question	Demande reçue.
answer	Texte généré par le modèle, ou refus.
sources	Événements récupérés, dédupliqués par UID ; pas la preuve que tous sont cités.

Champ	Interprétation
retrieved_chunk_count	Nombre de passages fournis au modèle, distinct du nombre de sources uniques. Ce champ n'était pas conservé dans le rapport de démo historique et n'est pas inventé dans sa relecture.

Rester sur le support et la démo

Ton déroulé utilise seulement le diaporama et cette page de démonstration. Après les deux scénarios, reviens à la slide 9 ; termine sur les résultats, les limites et la conclusion. Tu n'as pas de visite de GitHub, du code ou du rapport à effectuer dans le récit prévu.

Le rapport et le dépôt restent des livrables disponibles si le jury demande à les consulter. Le brief prévoit leur présentation : les résultats, les choix et l'architecture sont expliqués sur tes slides ; une demande explicite de consultation pourra nécessiter de montrer le document concerné.

Les réponses aux questions de JérémY

Écoute la question jusqu'au bout. Donne d'abord la réponse courte, puis développe si nécessaire. Quand tu ne sais pas, distingue ce qui a été vérifié de ce qui demande une mesure.

▼ Pourquoi le RAG, et avez-vous entraîné le modèle ?

Le RAG ajoute les passages du catalogue au moment de la question. Je n'ai ni entraîné ni ajusté Mistral ; j'ai construit un index et un prompt.

Les données peuvent évoluer sans réentraîner le modèle : on recollecte puis on réindexe. Le RAG ne supprime pas les erreurs de génération.

Preuve : rag.py : answer_question()

▼ Pourquoi Mistral et ces deux modèles ?

Mistral est demandé par le brief. mistral-embed sert à rechercher. Le POC évalué utilisait mistral-small-latest ; la démo utilise ministral-8b-2512 car Small était limité aujourd'hui.

Le modèle alternatif a réussi les deux scénarios API réels. Ce n'est pas un benchmark comparatif et les scores RAGAS de juillet restent ceux de Small. Comme les embeddings n'ont pas changé, aucun réindexage n'est nécessaire. Le nom de version est fixé pour cette démo.

Preuve : settings.py ; rag.py ; indexing.py

▼ Pourquoi FAISS et quel index utilisez-vous ?

FAISS permet une recherche vectorielle locale sans serveur supplémentaire. L'index livré est IndexFlatL2, une recherche exacte par distance euclidienne au carré.

Il contient 14 903 vecteurs de 1 024 dimensions. Les dimensions ne sont pas des catégories nommées. Pour beaucoup plus de données, il faudrait mesurer les coûts de mémoire et de recherche avant de choisir un index approximatif ou une base serveur. Le code n'applique pas de normalisation explicite pour une recherche cosinus.

Preuve : seed-data/index/faiss/index.faiss ; indexing.py

▼ Pourquoi 1 000 caractères, 200 de chevauchement et top-k 4 ?

Ce sont les réglages de départ du POC : des passages assez courts, un chevauchement pour les frontières et quatre contextes pour limiter le

bruit.

Ils n'ont pas été prouvés optimaux. Je comparerais plusieurs tailles et valeurs de k en mesurant le rappel des événements attendus, la fidélité, la latence et le coût. RecursiveCharacterTextSplitter respecte les séparateurs ; ce n'est pas une découpe fixe tous les 800 caractères.

Preuve : index_manifest.json ; settings.py

▼ Vos événements sont-ils encore récents et à venir ?

Le seed couvre la première date du 10 juillet 2025 au 10 juillet 2026. C'était le périmètre construit en juillet ; il n'est pas maintenu à jour au 1er octobre.

Certains événements ont maintenant plus d'un an, et la borne haute exclut les nouveaux événements qui commencent après juillet. Quelques événements récurrents se prolongent après la borne. Pour remplir durablement le besoin d'événements à venir, il faut une collecte planifiée avec un horizon futur et un filtrage des occurrences au moment de la réponse. Je reconnais cet écart actuel.

Preuve : fetch_manifest.json ; ingestion.py : build_records_where_clause()

▼ Pourquoi Opendatasoft plutôt que l'API native OpenAgenda ?

La mission renvoie au dataset public OpenAgenda d'Opendatasoft ; c'est cette source qui est utilisée et documentée.

La collecte est sans clé sur cette source. Elle normalise un schéma qui pourrait être alimenté par une autre source. Je ne prétends pas utiliser les endpoints natifs OpenAgenda.

Preuve : ingestion.py : OpenAgendaClient

▼ Comment évitez-vous les hallucinations ?

Le prompt impose le contexte et le refus si l'information manque ; les sources permettent un contrôle. Cela réduit le risque sans garantir zéro erreur.

Je compléteraï par des filtres explicites, un seuil calibré, un contrôle des faits, des tests adverses et une revue humaine. Les descriptions externes doivent aussi être traitées comme des données non fiables pour éviter les injections de prompt.

Preuve : rag.py : SYSTEM_PROMPT

▼ Pourquoi Lyon retourne-t-il quand même des sources ?

Le top-k renvoie les voisins les plus proches, même si leur ville ne correspond pas. Ce sont des documents récupérés, pas forcément des

recommandations.

Le refus Lyon observé vient du modèle lisant le contexte Paris. Il n'existe pas de blocage géographique codé ni de seuil. Un filtre ville ou un rejet mesuré rendrait le comportement plus fiable et pourrait éviter un appel de génération inutile.

Preuve : rag.py : build_retriever(), build_source_entries()

▼ Que signifient 0,900 et 1,000 ?

0,900 mesure la fidélité au contexte sur trois cas. 1,000 mesure la précision et le classement des contextes par RAGAS, pas la récupération de tous les événements pertinents.

Ce ne sont ni une satisfaction de 90 % ni une exactitude universelle de 100 %. Il manque une mesure explicite de rappel du retrieval et un échantillon plus large. Les résultats datent du 23 juillet et peuvent varier avec le juge.

Preuve : ragas_results.json ; ragas_evaluation.py

▼ Pourquoi la correction n'est-elle que 0,683 ?

Une réponse peut être soutenue par un contexte mais s'écarter de la référence, oublier une information ou ajouter un détail fragile.

Je relis les cas plutôt que de déduire une cause unique du score. Le cas famille obtient environ 0,581 en correction : des horaires sont généralisés à plusieurs visites. Les références doivent être enrichies lorsque nécessaire, et les réponses doivent mieux distinguer les faits sourcés des conseils.

Preuve : rapport, section 7 ; ragas_results.json

▼ Quatre cas corrects : votre précision est-elle de 100 % ?

Non. Ce classement porte sur quatre exemples et sur une règle locale qui accepte un titre présent dans la réponse ou dans les sources.

La similarité locale est lexicale avec SequenceMatcher, pas sémantique. L'exact match est nul dans la sortie enregistrée malgré quatre classements corrects. Cette évaluation vérifie surtout des repères ; elle doit être complétée par les faits, les dates, le refus et des annotations plus nombreuses.

Preuve : evaluation.py : classify_result(), match_expected_titles()

▼ Pourquoi seulement trois exemples RAGAS ?

Ils démontrent une évaluation réellement exécutée avec les contextes exacts. Ils sont insuffisants pour conclure statistiquement.

J'ajouterais villes, catégories, dates, questions ambiguës, négations, hors corpus et paraphrases. Je séparerais les cas utilisés pour choisir les réglages de ceux utilisés pour les évaluer et j'intégrerais une revue humaine indépendante du juge.

Preuve : tests/fixtures/ragas_eval_dataset.csv

▼ Les 76 tests appellent-ils réellement Mistral ?

La suite de tests utilise des doublures pour vérifier le code sans dépendre du réseau. La démo API et l'évaluation enregistrée font les appels réels séparément.

Les tests ne prouvent ni la disponibilité du fournisseur ni la qualité de toutes les réponses. Les appels réels ont été vérifiés séparément aujourd'hui. Dix-sept régressions supplémentaires couvrent les erreurs fournisseur, l'arrêt des répétitions et la gestion des dates dans le contexte.

Preuve : tests/conftest.py ; test_rag.py ; test_api.py

▼ Est-ce réellement hors ligne ?

Non. Le corpus et FAISS sont locaux, mais Mistral est appelé pour l'embedding de la question et la génération.

Les pages et les résultats enregistrés fonctionnent hors ligne ; ils ne remplacent pas un RAG live. Une version entièrement locale demanderait des modèles locaux compatibles et un nouvel index si l'embedding change.

Preuve : rag.py ; indexing.py : _build_embeddings()

▼ Que fait /rebuild exactement ?

Il relit le Parquet, redécoupe, recalcule les embeddings et remplace l'index. Il invalide ensuite le retriever et le client de chat.

Il ne télécharge pas de nouveaux événements. Pour actualiser les données il faut d'abord fetch_events.py, puis l'indexation. Cela peut coûter et durer ; je ne le lance pas pendant la soutenance. En production, une reconstruction dans un dossier séparé puis une bascule atomique éviterait les conflits.

Preuve : service.py : rebuild() ; indexing.py : rebuild_index_artifacts()

▼ Comment protégez-vous la clé et les endpoints ?

La clé est dans l'environnement ou .env ignoré par Git. Docker ne la copie pas. /rebuild dispose d'un jeton optionnel.

Le jeton n'est pas obligatoire dans le POC ; avant exposition il faut l'activer, ajouter authentification et limites d'appels. Les erreurs fournisseur sont déjà filtrées. L'index pickle doit provenir d'une source de confiance. Je montre .env.example, jamais .env.

▼ Que garantit Docker et /health ?

Docker fournit le code, les dépendances et un seed dans un environnement commun. /health confirme que le serveur répond et expose sa configuration.

Health ne prouve pas que la clé fonctionne ni que l'index est chargé. Le chargement est paresseux à la première question. Le volume conserve l'index de travail. Une readiness complète devrait vérifier le corpus et un contrôle adapté du fournisseur, sans appels coûteux à chaque sondage.

Preuve : Dockerfile ; docker_entrypoint.sh ; service.py : health()

▼ Comment avez-vous débloqué Mistral ?

J'ai isolé les appels : les embeddings fonctionnaient, la génération avec Small renvoyait 429. Ministral 8B répondait ; je l'ai configuré puis vérifié les deux scénarios RAG.

La clé était valide et identique dans Docker. L'accès aux limites administratives n'était pas autorisé avec cette clé : la limite précise n'est pas connue. L'API traite désormais les erreurs fournisseur sans exposer leurs détails ; le script de démo ne relance plus 60 fois les appels. Il faudra réévaluer le modèle alternatif et surveiller les limites par modèle.

Preuve : Vérification du 1er octobre ; api.py : gestion des exceptions

▼ Avez-vous une CI/CD et mesuré la montée en charge ?

Non. Les scripts et les tests sont livrés ; aucune pipeline CI/CD ni benchmark de charge n'est implémenté.

Je proposerais tests sans secret à chaque push, construction Docker, contrôles de disponibilité, puis déploiement contrôlé. Je mesurerais p50/p95, débit, taux d'erreur, coûts et ressources sur une charge représentative. Une démo courte ne permet pas de les extrapoler.

Preuve : tests/ ; Dockerfile ; README.md

▼ Quel est le coût et la valeur pour Puls-Events ?

La valeur attendue est une découverte plus naturelle avec des sources vérifiables. Elle n'a pas encore été mesurée auprès d'utilisateurs.

Coût = embeddings de construction et mise à jour + embedding par question + tokens de génération, et éventuellement juge RAGAS. Je ne donne pas de prix non vérifié. Un pilote comparerait satisfaction, clic vers source, refus correct, latence et coût par demande à une recherche classique.

▼ Comment comparer un nouveau modèle d'embedding ?

Je reconstruis l'index avec ce modèle et j'utilise le même modèle pour les questions. On ne peut pas mélanger deux espaces vectoriels.

Même si les deux vecteurs ont la même dimension, leurs coordonnées ne sont pas forcément comparables. Je garde le même corpus et jeu d'évaluation pour comparer rappel, pertinence, coût et latence. Le manifeste trace le modèle utilisé.

Preuve : index_manifest.json ; service.py ; settings.py

▼ Comment rendre le système plus utile pour des dates et lieux précis ?

J'ajouterais des contraintes métier explicites sur la ville et les occurrences, en complément de la recherche sémantique.

Un événement récurrent ne doit pas être interprété comme une disponibilité continue entre first_timing et last_timing. Le corpus brut conserve timings ; le POC utilise surtout première et dernière dates. Un futur moteur doit sélectionner l'occurrence qui correspond à la demande et vérifier accessibilité, tarifs et public lorsque les données existent.

Preuve : ingestion.py : _extract_timings() ; métadonnées de FAISS

Répondre sans inventer et garder le fil

Si tu oublies un terme

Reviens au trajet : **catalogue** → **passages** → **sens** → **bibliothécaire** → **rédacteur** → **guichet**. Ensuite, pose-toi quatre questions : **qu'est-ce qui entre, que fait cette brique, qu'est-ce qui sort, pourquoi ce choix ?** Pour FAISS : vecteur de question → comparaison des distances → passages proches → recherche locale simple.

Si le jury te pose une question dont tu ignores la réponse

« Sur ce point, je préfère être précis : ce que j'ai vérifié dans le POC est... Pour le détail que vous demandez, je dois consulter le code ou mesurer avant de conclure. »

Si une erreur est pointée

Reconnais l'écart, explique son effet, donne la correction. Exemple : « Vous avez raison, une fenêtre figée en juillet n'est plus un catalogue à jour en octobre. Il faut une collecte planifiée et des filtres d'occurrences ; le mécanisme RAG peut être conservé. »

Si tu prends du retard

Utilise « L'idée à retenir » sur les fiches suivantes. Garde impérativement les données, les modèles, la démo, les résultats et les limites. Réduis les explications secondaires, puis conclus sur le support. Si tu es trop rapide, explique un exemple concret de chunk ou un score ; évite de meubler.

Si Docker ne répond plus

```
docker compose ps
docker compose up -d --no-build
```

Une seule relance. Si elle échoue, montre l'enregistrement et l'état réel ; ne passe pas plusieurs minutes à déboguer devant le jury.

Si le support HTML ne s'affiche plus

Depuis la racine du projet, lance `python scripts/lancer_soutenance.py` : il relance les pages en arrière-plan. Utilise sinon la présentation PDF ou PowerPoint synchronisée. Le guide peut aussi être imprimé. Tous les textes sont autonomes ; seuls les appels Mistral et GitHub nécessitent internet.

Ce que tu peux dire pendant le débrief

Quand l'évaluateur quitte le rôle de Jérémie, écoute son retour. Tu peux dire : « Le projet m'a appris à distinguer la qualité des données, la recherche et la fidélité de génération. La suite prioritaire est d'actualiser les données et d'élargir l'évaluation. » Adapte cette phrase à ce que tu as réellement compris et vécu.

MISSION ET PIÈCES LIÉES

Les exigences et les preuves disponibles

Cette préparation reprend la mission, ses six étapes, les consignes de soutenance et les dix rubriques du modèle de rapport fourni. Le document exporté contient aussi le cours RAG avec un cas municipal : ces exercices pédagogiques ne sont pas des livrables de la mission Puls-Events.

Exigence	Preuve	État / limite
RAG LangChain, Mistral, FAISS	rag.py, indexing.py, service.py	Livré ; génération distante.
Scripts de reconstruction	fetch_events.py, build_index.py ; /rebuild	Livrés ; rebuild relit le Parquet, sans collecte.
Zone et événements récents	Manifeste Paris, juillet 2025 → juillet 2026	Corpus figé ; certains événements dépassent un an aujourd'hui. Actualisation nécessaire.
API REST exploitable	/ask, /rebuild, /health, /docs	Serveur disponible ; /ask vérifié avec Ministral 8B après une limite sur Small.
Rapport technique	README + PDF existant	Couvre les choix, résultats, limites et perspectives ; rapport actualisé le 1er octobre.
Tests et jeu annoté	76 tests ; 4 cas locaux ; 3 cas RAGAS	Tests passent aujourd'hui ; petit échantillon métier.
Docker	Dockerfile, Compose, seed et volume	Image existante démarrée ; aucun nouveau build complet affirmé.
Démo live	Swagger et démonstrateur direct	Deux scénarios réussis en direct avec Ministral 8B ; enregistrement du jour disponible en secours.
Présentation 10 à 15 slides	12 slides HTML et export synchronisé	HTML pour présenter, formats de secours pour le brief.
Rapport et dépôt disponibles	Livrables remis ; résultats et architecture expliqués sur le support	Pas de visite prévue dans le récit ; consultation possible à la demande du jury.
ZIP et lien GitHub	Archive historique dans outputs ; dépôt hgbe-GH/Projet7	La nouvelle archive de soutenance rassemble les supports actuels ; code, preuves et supports inclus dans la mise à jour GitHub ; dépôt plateforme à effectuer.

Le modèle de rapport et la fiche d'autoévaluation

Objectifs → architecture → préparation/vectorisation → modèle NLP → base vectorielle → API → évaluation → perspectives → dépôt → annexes. Le PDF existant suit ce modèle. La fiche d'autoévaluation liée à la mission a également été consultée ; elle demande notamment un schéma UML clair, des tests relançables, des résultats documentés et une API accessible. Le complément ci-dessous fournit le schéma des composants et distingue les éléments vérifiés des points à compléter.

[Architecture détaillée et autoévaluation ↗](#)

Traçabilité des affirmations

Volumes et configuration : manifestes versionnés et inspection du seed. Comportement : lecture de rag.py, indexing.py, service.py et api.py. Tests : exécution du 1er octobre. Scores : ragas_results.json, généré le 23 juillet 2026 à 14:36:49 UTC. Démo du jour : demo_api_2026-10-01.json, deux scénarios, modèle, horodatage et durée globale conservés. Les scores ne sont ni extrapolés ni remplacés par des résultats inventés.

Guide et diaporama synchronisés sur les mêmes 12 étapes. Les supports précédents sont conservés dans archives/.